

Project #03, Part 01 (v1.0)

Assignment: Authentication microservice

Submission: via Gradescope

Policy: team-oriented, late work is accepted

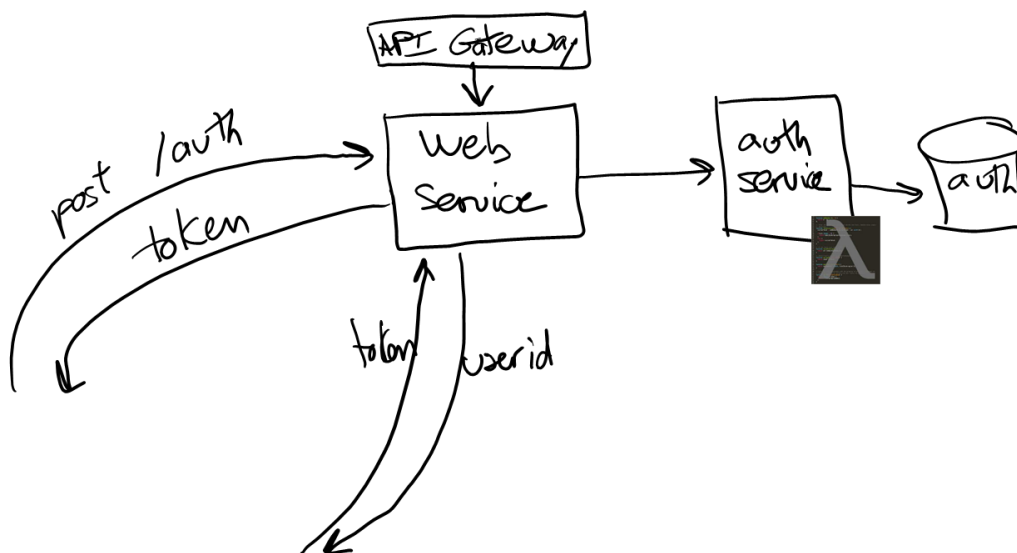
Complete By: Monday June 1st @ 11:59pm CST

Late submissions: see syllabus for late policy... No submissions accepted after Thurs 06/04 @ 11:59pm

Pre-requisites: Lab 04 (lambda, serverless computing)

Overview

In part 01 of Project 03 we're going to build an **authentication microservice** for user login and token verification. We'll use this service in part 02 to extend our **PhotoApp** to support modern authentication based on tokens. Here's a visual:



The microservice defines one API function **POST /auth**. Given a username and password, the service returns an *authentication token* if the user's password matches. Given an authentication token, the service returns the user's *userid* in the database if the token is valid. [*In true microservice design, the service is self-contained with its own web service, API, and database. This is generally the approach taken with systems that offer single sign-on, e.g. logging into NU's various infrastructure components such as email, Canvas, and Caesar.*]

CAVEAT: project 03 demonstrates an approach for authentication using passwords and access tokens. The

passwords and tokens are sent in CLEAR TEXT. It is assumed that in a real application, you would be protecting these security credentials by installing a server-side SSL/TLS certificate AND using HTTPS so that all network traffic is properly encrypted. Otherwise what we are demonstrating here is NOT SECURE, and should NOT be used unless you are incorporating a certificate and using HTTPS from the client.

Working as part of a team? You can form new teams in project 03...

As in earlier projects, you can work independently or in a team (max size 4), up to you. We are not assuming the same teams as projects 01 and 02, so If you plan to work to work in a team on project 03 please do the following:

1. Login to canvas
2. Click on "People"
3. Click on "Groups"
4. Scroll down to the groups named "**Project 03 1**", "**Project 03 2**", etc.
5. Pick a group that has no members, and add yourself to the group. Have your teammates add themselves to the same group.

That's it! Note that submissions will be collected via Gradescope, not Canvas. When you submit to Gradescope, only one team member needs to submit; every team member receives the same score.

Step 1: Creating the authentication service database

The first step is to create a new database named **authsvc** on your existing MySQL server running in RDS. The complete SQL to create the database is given below (and [dropbox](#)); recall you'll need to connect to MySQL using your **admin** account to create this database. The **authsvc** database consists of 2 tables, **users** and **tokens**:

```
USE sys;

CREATE DATABASE IF NOT EXISTS authsvc;

USE authsvc;

DROP TABLE IF EXISTS tokens;
DROP TABLE IF EXISTS users;

CREATE TABLE users
(
    userid      int not null AUTO_INCREMENT,
    username    varchar(64) not null,
    pdhash      varchar(256) not null,
    givenname   varchar(64) not null,
    familyname  varchar(64) not null,
    PRIMARY KEY (userid),
    UNIQUE      (username)
);
```

```

ALTER TABLE users AUTO_INCREMENT = 80001; -- starting value

CREATE TABLE tokens
(
    token            varchar(128) not null, -- authentication token
    userid           int not null,         -- user that token identifies
    expiration_utc    datetime not null,    -- token's expiration date/time UTC
    PRIMARY KEY (token),
    FOREIGN KEY (userid) REFERENCES users(userid)
);

--
-- Insert some users to start with:
--
-- PWD hashing: https://phppasswordhash.com/
--
INSERT INTO users(username, pwdhash, givenname, familyname) -- pwd = abc123!!
values('p_sarkar',
'$2y$10$/8B5evVyaHF.hxVx0i6dUe2JpW89EZno/VISnsiD1xSh6ZQsNMtXK',
'Pooja',
'Sarkar');

INSERT INTO users(username, pwdhash, givenname, familyname) -- pwd = abc456!!
values('e_ricci',
'$2y$10$F.FBSF4zlas/RpHAXqsuF.YbryKNr53AcKBR3CbP2KsgZyMxOI2z2',
'Emanuele',
'Ricci');

INSERT INTO users(username, pwdhash, givenname, familyname) -- pwd = abc789!!
values('l_chen',
'$2y$10$GmIzRsGKP7bd9MqH.mErmuKvZQ013kPfKkbeUAHxar5bn1vu9.sdK',
'Li',
'Chen');

--
-- Create fictitious users of database to limit access:
--
-- ref: https://dev.mysql.com/doc/refman/8.0/en/create-user.html
--

DROP USER IF EXISTS 'authsvc-read-only';
DROP USER IF EXISTS 'authsvc-read-write';

CREATE USER 'authsvc-read-only' IDENTIFIED BY 'abc123!!!';
CREATE USER 'authsvc-read-write' IDENTIFIED BY 'def456!!!';

GRANT SELECT, SHOW VIEW ON authsvc.*
TO 'authsvc-read-only';
GRANT SELECT, SHOW VIEW, INSERT, UPDATE, DELETE, DROP, CREATE, ALTER ON authsvc.*
TO 'authsvc-read-write';

FLUSH PRIVILEGES;

```

The *users* table is identical to projects 01 and 02, storing the userid, the user name, actual name, and the user's password as a one-way encrypted string. [*One-way encryption is often referred to as "hashing", and the one-way nature of hashing implies the original password cannot be recovered from the hash alone.*] For reference, here are the actual passwords used to generate the hashes in the database:

1. p_sarkar, **abc123!!**
2. e_ricci, **abc456!!**
3. l_chen, **abc789!!**

The *tokens* table stores the login tokens that authenticate users in API calls. Every login token is associated with a user, so **userid** is a foreign key into the **users** table. The **token** is the primary key, and thus guaranteed to be unique. By default tokens expire after 30 minutes (this is programmed into the authentication service but also parameterized).

Step 2: Additional Lambda function layers

In Lab 04 we created a Python layer for the *requests* module. Here in project 03 we need two additional layers: one for hashing passwords (one-way encryption), and one for executing SQL queries against MySQL. Here are the steps to build and install these two layers (these layers are also available in [dropbox](#) if you have problems creating them):

1. Open **CloudShell** by searching (or click the > icon on the title bar); if you had problems with CloudShell in Lab 04, try using a local terminal window or the cs310-client docker image.
2. Create a .zip file that contains the necessary layer for hashing passwords (**bcrypt-layer.zip**):

```
rm -rf python
mkdir python
cd python
pip3 install bcrypt -t .
cd ..
zip -r bcrypt-layer.zip python
```

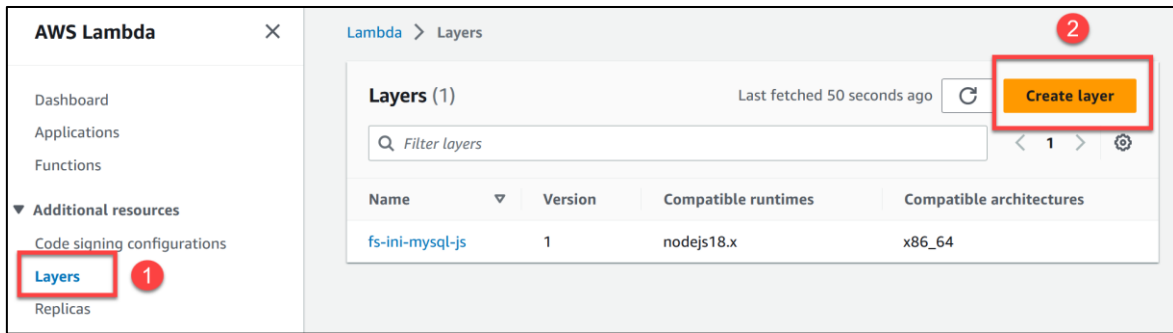
3. Copy your .zip file to one of your S3 buckets, which will make the file accessible to AWS Lambda:

```
aws s3 cp bcrypt-layer.zip s3://your-bucket-name
```

4. Find your .zip file in your S3 bucket, select to view properties, and copy the **Object URL**

Now, let's install the layer in Lambda so your Lambda functions can use it:

1. Open **Lambda**
2. Select Layers in the sidebar (under "Additional resources"), then press Create layer (see screenshot on next page):



3. Configure and create the layer:

- Set the name to **bcrypt-layer**.
- Select **Upload a file from Amazon S3**, then paste in the Object URL you copied earlier.
- Select **x86_64** under compatible architectures.
- Select the most recent Python version as the compatible runtime.
- Press **Create**!

We need another layer for accessing MySQL from within Lambda, in particular the **pymysql** module. Repeat the previous steps, starting from CloudShell and following the steps below. In this case, ignore any warnings / errors from the **pip3** command:

- ```
rm -rf python
mkdir python
cd python
pip3 install cryptography pymysql[rsa] typing_extensions==4.14.1 -t .
cd ..
zip -r pymysql-layer.zip python
```
- aws s3 cp pymysql-layer.zip s3://your-bucket-name**
- Open S3, find the .zip, and copy the **Object URL**
- Back in the lambda console, create a new layer “pymysql-layer”, point to the appropriate object URL for the requests-layer.zip in S3, configure with the same configuration options as we did for “bcrypt-layer”, and create.

## Step 3: Installing the authentication microservice

The Lambda function **authenticate** will represent our authentication microservice. It will have its own API, called via **POST /auth**. The function is passed either (1) a username and password, or (2) a token. When passed a username and password, the service checks the password against the *pwdhash* in the database. If the results match, then the authentication is successful and the service responds as follows:

- A status code of 200 is returned.
- The body of the message contains a login token (string) to identify the user in subsequent API calls.

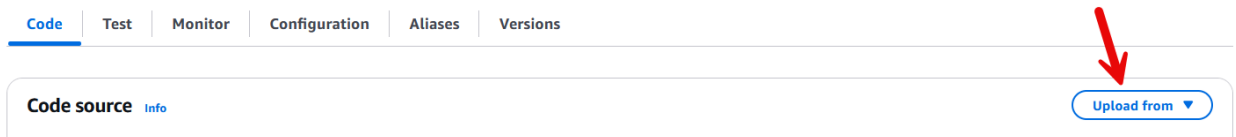
When passed a token, the service checks to make sure the token exists in the database, and confirms the token has not expired. If so, then the authentication is successful and the service responds as follows:

1. A status code of 200 is returned.
2. The body of the message contains the user's userid.

If authentication fails in either case, the service responds with a status code of 401, and a corresponding message is returned in the response body: "expired token", "invalid token", "invalid username", or "invalid password". [ **NOTE:** in a real system this would be considered too much information, and instead the service would merely respond with "authentication failed". ]

Starter code is provided, here are the steps to download and install the provided code:

1. Download "authenticate.zip" from [dropbox](#)
2. Login to AWS, install and create the **authenticate** lambda function:
  - a. Create a new lambda function named **authenticate**, selecting the most recent Python runtime and the architecture x86\_64.
  - b. You'll be dropped into the "Code" tab, select "Upload from" and upload the .zip:



- c. Open "authsvc-config.ini", and paste the RDS endpoint for **your** database server.
- d. Click "Deploy" ...
- e. Select "Configuration" tab, select "General configuration", click "Edit", and change the "Timeout" value to 5 minutes. Click save.
- f. In the file explorer pane you'll see some test files --- "test01.txt", etc. Open each text file, and copy the contents --- this is the Event JSON needed for a test call to this lambda function. Click the "Test" button, select "Create new test event", and paste the JSON and save the test case. Repeat for each test file.

Don't test anything yet, just install the test cases...

- g. After creation, scroll down to "Layers", click "Add a layer", choose "Custom layers", and select the "pymysql-pypdf" layer you built in project 03 (we need *pymysql* from this layer). Don't forget to select the version # from the dropdown --- it should be version 1.

Repeat this process, adding the "bcrypt" layer you built earlier. Select version 1 from the dropdown.

At this point you have the starter code installed. Scroll through the code for the lambda function and get a sense for what is given... The code eventually accesses the request body, and looks to see what was passed. If a token is passed, we lookup that token in the database to see if it exists --- if not, we respond with a 401. If

the token exists, we retrieve the userid and the expiration time (UTC). We then compare the current time to the expiration time, and respond with either 200 (and the userid), or 401.

Otherwise we were passed a username and password --- the password is sent in CLEAR TEXT. We lookup the username and password in the database; note that the password is properly stored in an encrypted (*hashed*) format. If the username does not exist, we respond with 401. If the username exists, we hash the given password and compare it to the hashed password from the database. If they don't match, we respond with 401. If they match, we generate a unique, hard to guess token, store this in the database with an expiration of at most 30 minutes, and respond with the token and status code 200.

**There are five TODO tasks in the code for you to complete. Work through those now...**

When ready, run **test02** to confirm that authentication fails for an incorrect password. Next, run **test01** to authenticate a valid user; if all is well, a token should be returned. Copy-paste the returned token, edit the JSON for test03, paste the token, save, and invoke --- **test03** should successfully authenticate. Wait 2 minutes for the token to expire, and run test03 again --- authentication should now fail. The service is working!

## Step 4: Build an API Gateway to your authentication service

At this point the **authenticate** Lambda function should be installed, tested, and working. Use API Gateway to create and deploy a new REST API such that **POST /auth** calls your *authenticate* function. Don't forget to enable "Lambda Proxy Integration" as part of the configuration. After you create the POST method and link to the lambda function, perform a quick test to make sure the gateway is configured properly. Scroll the tabs over to "Test" --- "Method request", "Integration request", ..., "Test" --- and paste the following into the text field under "Request body":

```
{
 "username": "p_sarkar",
 "password": "abc123!!",
 "duration": 2
}
```

If all is well, you should get a token back. Deploy the API when you're done. Grab the invoke URL and save this in a file named "authsvc-client-config.ini". The format of this config file should be as follows:

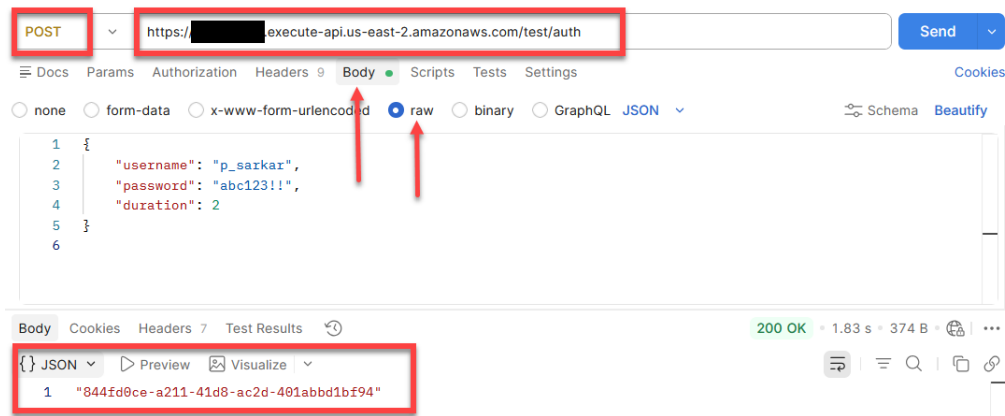
```
[client]
webservice=https://YOUR_GATEWAY_API.amazonaws.com
```

## Step 5: Client-side testing

The final step is to test your authentication service from the client side. The obvious way is to write some Python code and make a **requests.post()** call. Startup your client docker image, and mimic the code used in project 02 for calling the web service; in particular, see the "post\_image()" client-side API function defined in the file "photoapp.py". [ NOTE: the client docker image has a project03 folder, ignore/delete this folder. ]

Another approach commonly used by professional developers is **postman**, a freely-available tool online at

<https://postman.com>. Create a free account, and this tool allows you to test web services using all the supported HTTP verbs: GET, PUT, POST, DELETE, etc. If you want to give postman a try, signup at <https://postman.com>. Create a new workspace, and then click the "+" to create a new request. Change the verb to POST, paste the URL for the call (invoke URL + **/auth**), and set the body of the request to the same text we used to test from API Gateway:



Click the send button, and if all is well, you should get back a token. Your service is successfully deployed!

## Electronic Submission

Your work will be collected and graded using Gradescope. We are grading the deployment of your authentication service. You'll need to submit two configuration files:

1. **authsvc-config.ini** containing your RDS configuration information (download from Lambda)
2. **authsvc-client-config.ini** created in step 4 to denote your authentication service

If you are working in Docker (e.g. testing via a client-side app), you can submit your config files as follows:

```
/gradescope/gs submit 1288073 8159384 authsvc-config.ini authsvc-client-config.ini
```

Your goal is an autograder score of 50/50; you have unlimited submissions. If the above fails, then gradescope is not yet accepting submissions; come back and try again later. If you are working outside Docker, you can submit from your LOCAL computer by viewing your folder locally using Mac/Windows/Linux, open Gradescope site "Project 03, part 01 - authsvc" and drag-drop your two .ini files.

By default, Gradescope records the score of your **LAST SUBMISSION** (not your highest submission score). If you want us to grade a different submission, activate the appropriate submission via your "Submission History". This must be done before the due date.